TECHNICAL SYSTEM GUIDE

ELASTIQ retail pricing decision studio

A step-by-step reference for analysts, data scientists, engineers, and technical stakeholders. Every functionality is explained in three layers: at a glance, how it works, and full technical detail.

Use Level 1 for orientation, Level 2 for implementation understanding, and Level 3 for model, data, control, and engineering details.

HOW TO USE THIS GUIDE

System orientation

Every stage produces evidence or a gate. Only Optimize selects a price.

The system has a Python evidence pipeline and a React/Vite decision workbench. The pipeline produces validated model evidence. The workbench allows reviewed scenario execution, comparison, and export. The final price is selected only after causal, forecast, feasibility, and validation controls are applied.

Layer	Purpose	Main implementation
Source and validation	Establish reliable analytical inputs	CSV sources, pandas, schema and business-rule checks
Evidence models	Estimate causal price response, promotion readiness, and future demand	statsmodels, linearmodels, scikit-learn, XGBoost
Market context	Structure recent relevant unstructured evidence	TF-IDF retrieval, metadata and date filters
Decision engine	Evaluate feasible prices and commercial objectives	NumPy/SciPy Python engine and independent JavaScript engine
Review surface	Configure, compare, explain, and export	React, Vite, CSV, jsPDF
Assurance	Detect regressions and unsafe outputs	pytest, browser validation suite, parity fixtures, build and audit

Design rule: evidence and decision are separated. A model may produce a score, but only a passed readiness gate may allow that score to affect price.

DEVELOPER ORIENTATION

Repository map and execution order

Location	Responsibility
start.bat / start.sh	One-click startup and optional full data refresh
scripts/run_pipeline.py	Cross-platform ordered execution of stages 01 through 08
src/pipelines	Stage orchestration, input/output wiring, and saved artifacts
src/models	Elasticity, causal, uplift, forecasting, and retrieval logic
src/optimization	Demand response, constraints, price search, and decision validation
src/data / src/features	Splitting, validation, cleaning, aliases, and feature construction
app/frontend/src	React workbench, JavaScript engine, live lab, input profiler, reports, and demo data
app/frontend/src/workers	Isolated live experiment execution and progress events
tests	Python unit, regression, plotting, validation, and parity fixtures
data/processed	Stage outputs and final recommendation evidence
output/pdf	Share-ready high-level and technical documentation

Execution is strictly ordered because each stage consumes saved products of an earlier stage. The final stage checks its upstream dependencies before optimization. Browser data can be refreshed only after the Python outputs and parity fixture are regenerated.

Normal team use does not require Python. Run the standard launcher with packaged data. Use --refresh-data only when the analytical outputs must be recomputed.

FUNCTIONALITY 1 OF 15

Application startup and runtime

Give Windows, Linux, and macOS users one predictable way to start the workbench and an optional way to refresh analytical outputs.

LEVEL 1 - AT A GLANCE

Run `start.bat` on Windows or `./start.sh` on Linux/macOS. The browser opens at localhost and the terminal remains the application control window.

LEVEL 2 - HOW IT WORKS

The launcher checks Node.js and npm, creates a local runtime folder, installs exact web dependencies only when needed, starts Vite on 127.0.0.1, and opens the default browser. `--refresh-data` also provisions a Python virtual environment and executes the eight stages.

LEVEL 3 - FULL TECHNICAL DETAIL

Runtime state is isolated under `.runtime`; npm uses a project-local cache. `scripts/run_pipeline.py` calls every stage with the active interpreter and prepends the repository root to PYTHONPATH. A non-zero subprocess status stops the refresh. The launcher does not expose the service beyond localhost.

Inputs	Outputs
Node.js 18+; optional Python 3.10+	Local Vite workbench; optionally refreshed CSV/model artifacts

Primary controls: Dependency checks, exact `npm ci`, local-only host binding, fail-fast process status, Ctrl+C shutdown.

FUNCTIONALITY 2 OF 15

Input data profiling and initial analysis

Explain the exact decision rows before optimization and surface structural, commercial, capacity, or evidence-readiness concerns early.

LEVEL 1 - AT A GLANCE

Shows what data is loaded, whether it is complete, and what the portfolio looks like before a price is calculated.

LEVEL 2 - HOW IT WORKS

Profiles decision-unit identity, categories, stores, products, numeric completeness, invalid relationships, duplicate IDs, price distribution, demand, margin, competitor position, inventory pressure, elasticity range, causal coverage, and promotion readiness. A category view identifies where demand, risk, and evidence are concentrated.

LEVEL 3 - FULL TECHNICAL DETAIL

`analyzeInput` evaluates nine core numeric fields, finite-value coverage, price/cost and sign rules, unique `itemId` values, and evidence provenance. It calculates min, median, mean, and total statistics using finite values only. Inventory pressure means available stock is at or below 105% of baseline demand. Causal readiness recognizes the explicit flag or approved IV source; category aggregates retain items, average price and margin, demand, inventory-risk rate, causal rate, and promotion-readiness rate. React memoization recalculates the profile whenever input rows change.

Inputs	Outputs
Packaged, generated, edited, or CSV-imported decision rows	Portfolio and category input profile, completeness and readiness indicators

Primary controls: Finite-value checks, required commercial relationships, duplicate detection, explicit causal provenance, transparent readiness definitions.

FUNCTIONALITY 3 OF 15

01 - Data validation

Prevent missing, duplicate, malformed, or commercially inconsistent records from contaminating models and decisions.

LEVEL 1 - AT A GLANCE

Checks whether transaction and market-note data are complete and sensible before any analysis begins.

LEVEL 2 - HOW IT WORKS

Validates required fields, numeric types, missing values, duplicates, date coverage, price and cost relationships, inventory consistency, revenue arithmetic, and permitted ranges. It writes a clean dataset plus a quality summary and consistency audit.

LEVEL 3 - FULL TECHNICAL DETAIL

The stage loads configured raw sources, applies column aliases, coerces dates and numerics, and separates critical checks from review warnings. Cross-field rules include selling price versus regular price, units versus inventory, unit cost versus selling price, and recomputed revenue/profit tolerances. A manifest records the stage inputs and outputs.

Inputs	Outputs
Raw retail transactions and market notes	Clean retail data, clean notes, overview, quality summary, audit, manifest

Primary controls: Required-schema checks, critical/review severity, deterministic output paths, explicit row counts and date range.

FUNCTIONALITY 4 OF 15

02 - Exploratory analysis

Describe where demand, revenue, margin, discounting, promotion, and stock risk are concentrated before modelling.

LEVEL 1 - AT A GLANCE

Turns validated rows into business KPIs and portfolio views that explain what is happening.

LEVEL 2 - HOW IT WORKS

Calculates overall KPIs and summaries by category, product, store, region, discount band, and date. It visualizes revenue, units, margin, price position, promotion response, stockouts, and calendar patterns.

LEVEL 3 - FULL TECHNICAL DETAIL

Feature engineering adds margin, discount, calendar, price-position, and stockout fields. Aggregations retain observations, quantities, revenue, costs, profit, and rates. Correlation outputs are descriptive only and are not passed as causal price sensitivity. Figures and CSV summaries are reproducible from the validated dataset.

Inputs	Outputs
Validated retail dataset	KPI tables, segment summaries, correlations, and commercial figures

Primary controls: Stable grouping keys, safe divisions, transparent aggregation definitions, descriptive-versus-causal separation.

FUNCTIONALITY 5 OF 15

03 - Descriptive price elasticity

Measure the observed relationship between price and demand and provide diagnostics for the causal stage.

LEVEL 1 - AT A GLANCE

Estimates how much demand moved when price changed in the historical data.

LEVEL 2 - HOW IT WORKS

Fits log-log regressions by category and product, classifies elastic versus inelastic response, reports confidence and significance, shrinks noisy product estimates toward category baselines, and simulates price scenarios.

LEVEL 3 - FULL TECHNICAL DETAIL

The coefficient on log price is elasticity: a value of -1.5 means a 1% price rise is associated with about a 1.5% demand decline. Controls reduce observed confounding but do not solve price endogeneity. Product estimates carry status, observations, standard errors, confidence intervals, p-values, reliability, and shrinkage weight. This stage is diagnostic; executable pricing requires the causal gate.

Inputs	Outputs
Clean demand, price, product, category, store, promotion, and calendar fields	Category/product elasticity estimates and scenario simulations

Primary controls: Minimum observations and price variation, finite logs, confidence diagnostics, plausible-sign flags, no direct executable pricing.

FUNCTIONALITY 6 OF 15

04 - Causal price analysis

Estimate demand response caused by price rather than price changes that reacted to expected demand.

LEVEL 1 - AT A GLANCE

Checks whether the price effect is credible enough to support a recommendation.

LEVEL 2 - HOW IT WORKS

Uses two-stage least squares: cost-side instruments predict price in the first stage, then instrumented price estimates demand response in the second. It produces pooled and product-level estimates with strength and reliability diagnostics.

LEVEL 3 - FULL TECHNICAL DETAIL

The endogenous regressor is log selling price; the outcome is log units. Declared observed cost instruments include supplier and shipping indices, while exogenous controls and store effects account for measured context. Product models drop constant controls to avoid singular matrices. Reliability requires sufficient observations, an economically plausible sign, statistical support, and a strong first stage. Missing instruments cause an explicit failure; they are never fabricated.

Inputs	Outputs
Validated price, demand, controls, and observed instruments	IV estimates, OLS comparison, first-stage diagnostics, reliability fields

Primary controls: Observed-instrument requirement, first-stage F and partial R-squared, confidence intervals, per-product failure rows, causal provenance.

FUNCTIONALITY 7 OF 15

05 - Promotion uplift and readiness

Estimate incremental demand from promotions without treating normal buyers or leaked treatment outcomes as uplift.

LEVEL 1 - AT A GLANCE

Determines whether a promotion is likely to create extra demand and whether the evidence is safe to use in pricing.

LEVEL 2 - HOW IT WORKS

Builds a promotion propensity model, restricts analysis to common support, estimates an inverse-propensity weighted treatment effect, trains a two-model uplift learner, evaluates ranking on future dates, and produces segment summaries and opportunities.

LEVEL 3 - FULL TECHNICAL DETAIL

Only pretreatment features enter the model; selling price, competitor price, inventory, and stockout outcomes are excluded. Evaluation uses complete-date holdout partitions. Readiness combines propensity overlap, observed stockout censoring, ranking direction, and configured thresholds. Model scores can be stored for analysis even when the pricing-eligibility flag is false. Step 08 then applies zero promotion uplift.

Inputs	Outputs
Promotion flag, pretreatment product/store/calendar/context features, demand outcome	Propensity diagnostics, uplift predictions, deciles, metrics, readiness flag

Primary controls: Pretreatment-only schema, common support, date holdout, censoring threshold, ranking-quality gate, action-eligibility flag.

FUNCTIONALITY 8 OF 15

06 - Demand forecasting

Create the next-period quantity baseline against which every candidate price is compared.

LEVEL 1 - AT A GLANCE

Forecasts how many units are expected if the current pricing plan continues.

LEVEL 2 - HOW IT WORKS

Creates lags, rolling histories, price/promotion/calendar features, splits complete dates into train, validation, and test periods, tunes and trains XGBoost, compares it with naive forecasts, and generates one future row per store-product decision unit.

LEVEL 3 - FULL TECHNICAL DETAIL

All rows sharing a calendar date remain in one partition. Lags and rolling means are backward looking. Internal model selection uses only training/validation dates, while the final report uses a later untouched test block. Metrics include MAE, RMSE, R-squared, MAPE, WAPE, and SMAPE. Next-period construction advances the date explicitly, clears stale promotions, uses trailing or known context, and records forecast provenance.

Inputs	Outputs
Clean chronological demand history and known/trailing covariates	Test predictions, metrics, baselines, feature importance, next-period forecast

Primary controls: Unique-date splits, no future-derived features, naive benchmarks, explicit forecast date/source, complete store-product coverage.

FUNCTIONALITY 9 OF 15

07 - Time-safe market evidence

Convert recent competitor and customer notes into structured context without using future information or letting text directly choose a price.

LEVEL 1 - AT A GLANCE

Adds relevant recent market context beside each next-period decision.

LEVEL 2 - HOW IT WORKS

Cleans and chunks notes, indexes them with TF-IDF, retrieves the most relevant evidence for each decision row, extracts structured signals, and merges them with the forecast dataset.

LEVEL 3 - FULL TECHNICAL DETAIL

Eligibility requires note date <= decision date, a configured 120-day lookback, and exact normalized category and region matches when metadata is supplied. Similarity filtering and top-k retrieval limit evidence volume. Structured fields summarize sentiment, competitor position, sensitivity, promotion interest, trends, counts, and evidence dates. A bounded weighted market adjustment may affect demand; free text never selects price directly. Variance diagnostics flag constant features.

Inputs	Outputs
Dated market notes plus next-period decision rows	Document index, retrieved evidence, structured features, coverage and configuration

Primary controls: As-of-time boundary, lookback, exact metadata filters, minimum similarity, bounded adjustment, evidence provenance.

FUNCTIONALITY 10 OF 15

08 - Price optimization

Select the best feasible next-period price under the configured commercial objective and guardrails.

LEVEL 1 - AT A GLANCE

Compares allowed prices and recommends raise, hold, lower, or not scored.

LEVEL 2 - HOW IT WORKS

Combines forecast baseline, causal elasticity, unit cost, inventory, competitor context, eligible promotion uplift, market adjustment, and constraints. It estimates candidate demand, revenue, and profit, rejects infeasible candidates, selects the strongest objective value, and produces item and portfolio summaries.

LEVEL 3 - FULL TECHNICAL DETAIL

Demand follows Q(P)=Q0*(P/P0)^e. Realized sales are capped at available inventory in cap mode. Profit is (P-cost)*realized sales minus promotion cost. Current and candidate cases use identical inventory and promotion semantics. Causal elasticity precedence is explicit and non-causal rows hold by default. Cross-price effects are excluded until a true joint portfolio solver is used. If the current price is feasible, no method may return a worse configured objective.

Inputs	Outputs
Forecasts, causal elasticity, cost, inventory, competitor, promotion readiness, market features, policy	Recommendations, action/category/portfolio summaries, sensitivity scenarios

Primary controls: Price bounds, margin, quantity, profit, competitor, inventory, implementation step, causal gate, no-harm fallback.

FUNCTIONALITY 11 OF 15

Independent recommendation validation

Recompute the selected decision outside the optimization search and make any control failure visible before review.

LEVEL 1 - AT A GLANCE

Checks that every priced row really follows all configured rules.

LEVEL 2 - HOW IT WORKS

Recalculates price movement, demand response, margin, profit, competitor position, inventory treatment, implementation caps, evidence status, and objective performance for the final selected price.

LEVEL 3 - FULL TECHNICAL DETAIL

Validation distinguishes priced rows from explicit holds. Each control receives its own boolean field, while `meets_all_constraints` is the conjunction for scored rows. Tolerances handle numerical rounding without masking material failures. The validator verifies objective no-harm against a feasible current-price baseline and summarizes total, scored, held, passed, and failed rows. Release logic should stop when a scored row fails.

Inputs	Outputs
Final recommendation rows plus the same policy and source evidence	Row-level control fields and portfolio validation summary

Primary controls: Independent recomputation, explicit not-scored count, per-rule status, pass-rate summary, release-stop signal.

FUNCTIONALITY 12 OF 15

Browser decision workbench

Give commercial users an interactive surface to load data, configure policy, compare methods, review decisions, and export results.

LEVEL 1 - AT A GLANCE

A local browser application for running and explaining retail pricing scenarios.

LEVEL 2 - HOW IT WORKS

The React/Vite workbench presents data, configuration, recommendations, portfolio KPIs, action queues, scenario comparisons, category contributions, and evidence/constraint status. Users can switch among six optimization techniques under one common policy.

LEVEL 3 - FULL TECHNICAL DETAIL

The JavaScript engine independently implements the demand and economics model. Methods include grid, closed form, robust, Bayesian, multi-objective, and Lagrangian approaches. Shared fixtures enforce parity with Python for candidate demand, price grids, and rounding. Technique validation requires plausible economics, bounds, feasibility, no harm, and distinct price vectors. Manual rows have non-causal provenance and are held unless the policy explicitly changes.

Inputs	Outputs
Packaged demo data or validated CSV plus user configuration	Interactive decisions, summaries, scenarios, CSV and PDF exports

Primary controls: Causal provenance gate, shared guardrails, cross-language parity, method distinctness, explicit error handling.

FUNCTIONALITY 13 OF 15

Live scalable experiments and hybrid execution

Run real optimization workloads of different sizes, expose their execution process, and compare or combine methods under one frozen configuration.

LEVEL 1 - AT A GLANCE

Generates live examples, performs substantial real analysis, shows each computation stage and keeps every completed run available for review.

LEVEL 2 - HOW IT WORKS

The application opens on the Deep Analysis Runner. Users select Small (50), Medium (250), Large (1,000), or 5-2,000 custom units; choose a scenario profile, analysis depth and methods; and run comparison or hybrid execution. The six-stage monitor follows Start/profile, Optimize/refine, Stress test, Simulate risk, Validate, and Finish. It shows exact engine counters, measured time, events, charts, stress/risk tables, consensus, controls, top decisions and history.

LEVEL 3 - FULL TECHNICAL DETAIL

`scenarioGenerator.js` creates deterministic causal-provenance rows. `deepAnalysis.js` freezes a workload plan. Deep uses 1%, 0.5% and 0.25% grids, size-adjusted shocked-market reruns and 1,000 draws per method; Research adds 0.125%, more stress states and 3,000 draws. Stress states perturb demand, cost, elasticity, inventory, competitor price and market signal, then fully re-optimize every selected technique. Monte Carlo keeps each recommendation fixed while drawing portfolio and item uncertainty, producing P05/P50/P95, volatility and positive-uplift probability. `engine.js` directly counts portfolio optimizations, candidate evaluations, demand/profit evaluations and capacity iterations. Worker yields occur only after completed work; no delay is inserted. Release QA measured Deep all-method cases at 5.4 seconds/16.2M candidates for 50 units, 9.2 seconds/28.6M for 250, and 26.9 seconds/62.7M for 1,000 on the build environment. The final record retains convergence, all stress scenarios, risk percentiles, consensus, counters, method/hybrid decisions, configuration and timing; JSON and embedded-font PDF exports support review.

Inputs	Outputs
Generated size/profile/seed, analysis depth, selected techniques, active policy	Live computation stages, exact work counts, stress/risk distributions, decisions, JSON and PDF reports

Primary controls: Seed reproducibility, frozen workload, identical comparison input, exact instrumentation, worker isolation, row-level validation, deterministic hybrid tie-break, cancellable execution.

FUNCTIONALITY 14 OF 15

CSV import, reporting, and provenance

Move data and decisions in and out of the workbench without corrupting fields or losing the evidence behind a price.

LEVEL 1 - AT A GLANCE

Imports retail rows and exports review-ready CSV and PDF decision packages.

LEVEL 2 - HOW IT WORKS

The importer supports quoted CSV values and pipeline column aliases, validates required numerics and business relationships, and labels data origin. Exports include recommendation, scenario, category, configuration, control, and governance sections.

LEVEL 3 - FULL TECHNICAL DETAIL

The CSV parser handles escaped quotes, embedded commas, carriage returns, and blank lines. Aliases map pipeline forecast, cost, inventory, competitor, elasticity, promotion, market, and provenance fields into the browser schema. Validation rejects non-finite or impossible values and upward-sloping actionable elasticity. PDF libraries load only when requested, keeping the main bundle smaller. Exports preserve causal source, forecast source, promotion readiness, market evidence, status, and constraint flags.

Inputs	Outputs
CSV files or in-memory decisions and configuration	Validated browser rows, CSV recommendations, two-page decision PDF

Primary controls: Quote-aware parsing, aliases, strict numeric checks, provenance defaults, export error handling, lazy PDF loading.

FUNCTIONALITY 15 OF 15

Testing, build, and operational assurance

Detect mathematical, behavioural, integration, reporting, and dependency regressions before the application is shared.

LEVEL 1 - AT A GLANCE

Runs automated checks for the analytical pipeline and browser application.

LEVEL 2 - HOW IT WORKS

Python tests cover configuration, features, models, optimization, plots, text processing, validation, and known regressions. JavaScript tests exercise all six techniques, controls, advanced features, CSV behavior, provenance, method differentiation, multi-size generation, input profiling, live execution, hybrid selection, and Python parity. The frontend production build and dependency audit complete the release check.

LEVEL 3 - FULL TECHNICAL DETAIL

The current suite contains 358 Python tests. Regression cases include complete-date isolation, time-safe retrieval, instrument availability, known-truth elasticity recovery, inventory-baseline consistency, non-causal hold behavior, and objective no harm. Browser checks generate 50, 250, and 1,000 unit scenarios reproducibly; execute multiple techniques; validate timing and row controls; and confirm the hybrid method mix covers every item. Manual benchmarks run all six methods plus hybrid on Small, Medium, and Large cases. Shared fixtures compare four candidate cases, three grids, and four rounding cases. `npm run build` verifies bundling, Web Worker, and dynamic PDF chunks; `npm audit` checks the lockfile dependency graph.

Inputs	Outputs
Source code, fixtures, synthetic data, package lock	Pass/fail evidence, parity output, production bundle, audit status

Primary controls: Fail-fast exit codes, committed fixtures, behavioural assertions, known-truth tests, build and dependency checks.

REFERENCE

Core schemas and decision provenance

Domain	Required or important fields	Purpose
Decision identity	decision_date, store_id, product_id, category, region	Defines the store-product-time unit and retrieval filters
Commercial baseline	current_price, unit_cost, baseline_quantity, available_inventory	Defines current economics and next-period capacity
Causal evidence	elasticity, elasticity_source, elasticity_is_causal, first_stage_f	Defines demand response and whether pricing is allowed
Forecast evidence	forecast_quantity, forecast_date, forecast_source	Defines Q0 at P0 for the next period
Promotion evidence	promotion_uplift, promotion_readiness, promotion_cost	Allows or suppresses incremental demand and cost
Market evidence	market_adjustment, evidence_count, evidence dates/filters	Provides bounded, time-safe context
Policy	objective, price bands, margin, quantity, profit, competitor, inventory, step limits	Defines the feasible decision set
Decision	recommended_price, expected_quantity, revenue, profit, deltas, action	Communicates the selected scenario
Audit	status, reason, per-constraint flags, meets_all_constraints	Preserves failure behavior and release readiness

Every exported row should be self-describing: what was recommended, what baseline and model evidence supported it, which policy was active, which constraints bound, and whether independent validation passed.

CORE EQUATIONS

Mathematical reference

Constant-elasticity demand

Q(P) = Q0 x (P / P0)^e, where Q0 is the next-period baseline at current price P0 and e is causal price elasticity.

Inventory realization

Q_realized(P) = min(Q(P), available inventory) in cap mode. Constraint mode instead rejects scenarios whose unconstrained demand exceeds the permitted inventory condition.

Revenue and profit

Revenue(P) = P x Q_realized(P). Profit(P) = (P - unit cost) x Q_realized(P) - promotion cost.

Two-stage least squares

Stage 1 predicts log price from observed instruments and controls. Stage 2 regresses log quantity on predicted log price and controls. The log-price coefficient is the IV elasticity.

Inverse propensity weighting

Promotion observations receive weights based on treatment propensity so the weighted promoted and control groups are more comparable inside common support.

Forecast metrics

MAE averages absolute unit error. RMSE emphasizes large errors. MAPE averages row-level percentage error. WAPE divides total absolute error by total actual demand. SMAPE symmetrizes percentage error.

No-harm rule

When the current price is feasible, objective(recommended) must be greater than or equal to objective(current), within numerical tolerance.

Point estimates are used for the packaged demonstration. A live deployment should propagate forecast and elasticity uncertainty into downside-aware objectives and monitoring.

START, VALIDATE, STOP, AND TROUBLESHOOT

Operational runbook

Task	Windows	Linux/macOS
Start with packaged data	Double-click start.bat	chmod +x start.sh; ./start.sh
Refresh models and start	start.bat --refresh-data	./start.sh --refresh-data
Start without opening browser	start.bat --no-browser	./start.sh --no-browser
Stop	Press Ctrl+C in the command window	Press Ctrl+C in the terminal
Python tests	python -m pytest	python3 -m pytest
Browser tests	cd app\frontend && npm test	cd app/frontend && npm test
Production build	cd app\frontend && npm run build	cd app/frontend && npm run build

First-launch installation requires access to the npm registry. Data refresh also requires Python package installation. Runtime caches and the Python environment remain inside `.runtime`, which can be removed to force a clean setup.

Symptom	Likely cause	Resolution
Node.js is not installed	Node/npm unavailable on PATH	Install Node.js 18+ and reopen the terminal
Port 5173 is busy	Another Vite process is running	Stop the earlier terminal or use the displayed alternate port
Python refresh fails	Python or build dependency missing	Run without --refresh-data or install Python 3.10+ and retry
CSV is rejected	Missing, nonnumeric, impossible, or non-causal fields	Review the import error and correct the identified column
Item is held	Evidence or feasibility gate failed	Read status/reason; do not force a price without resolving the evidence

TEAM HANDOFF

Release and governance checklist

Area	Ready condition
Application	One-click launcher starts locally; browser engine validation and production build pass
Data	Source contract, freshness, row counts, dates, critical rules, and lineage pass
Causal model	Observed instruments, sufficient strength, plausible sign, stability, and business exclusion logic pass
Forecast	Unique-date evaluation, naive comparison, segment errors, freshness, and drift thresholds pass
Promotion	Overlap, censoring, ranking, and incremental-value thresholds pass before uplift is enabled
Market evidence	As-of-time, lookback, metadata matching, variance, and provenance pass
Optimization	Policy approved; current/candidate semantics match; no-harm and independent audit pass
Operations	Roles, approvals, audit retention, monitoring, alerting, rollback, and incident owners are defined
Pilot	Eligible scope and holdout are predefined; realized customer and financial outcomes are measured

The packaged system is ready for demonstration, training, reviewed scenario analysis, and controlled pilot preparation. Direct live price publishing requires the operational controls listed above.